RAKSHAK 2.0

RAKSHAK 2.0

Team: sshariharan82 | **Leader:** Hariharan S | **College:** St. Joseph's College of Engineering, Mail: sshariharan82@gmail.com |

Problem Statement — The Knowledge-to-Action Gap

Core Problem

Organizations have brand guidelines, tone rules, terminology standards, and legal disclaimers — but they live in **static PDFs, wikis, and Figma libraries** that creators never open while writing. The rules are not machine-readable, not queryable, and not enforced at creation time.

Three Structural Taxes Every Team Pays

Tax	What It Looks Like	Source
Search Tax	Copywriter hunts 40-page PDF for "latest disclaimer" — finds 3 versions, guesses	Interview observation (3/5 practitioners)
Error Tax	Wrong logo stretch, banned phrase ("guaranteed ROI"), missing CAN-SPAM footer ships to review	Interview observation (4/5 practitioners)
Latency Tax	MLR/brand/legal queue forms; same rule checked repeatedly; reviewers burn out	Interview observation (2/5 practitioners)

First User & Exact Workflow

User: Marketing reviewer at small-mid org (startup, agency, internal team) publishing 5–20 pieces/week. No dedicated brand/compliance team. Uses Google Docs, Notion, Canva, LinkedIn, CMS.

Moment of Failure: They paste a draft into a tool — any tool — and need to know *right now*: "Does this violate our rules? Which rule? What's the fix?"

Current Options All Fail for This User:

Solution	Why It Fails
Brand portals (Bynder, Frontify)	Repositories only. No scanning, no fixes, no integration. $50k+/yr (vendor pricing pages).
AI writers (Writer, Jasper, Grammarly)	Cloud = data leaves. Generic rules. Hallucinate citations. No diff UI.
ChatGPT/Custom GPT	IP leak. No deterministic layer. No rule citation. No memory.
MLR platforms (Veeva)	Pharma-only. Routes for approval — doesn't check while writing.

What the User Actually Needs

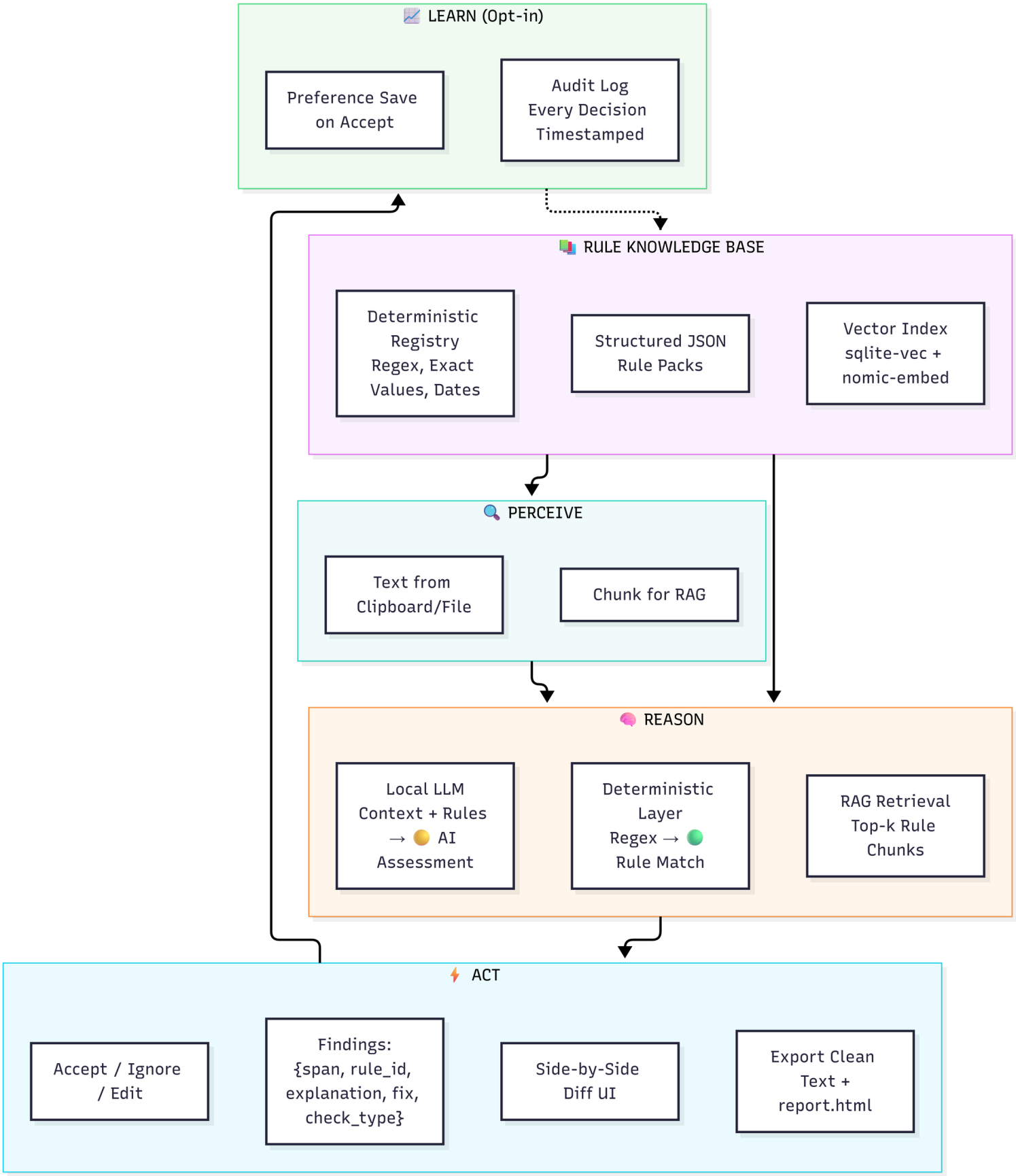
A **standalone local checker** that accepts pasted text or files, loads their rules, scans the draft, returns **cited findings with fixes**, lets them **accept/ignore each**, exports clean text + audit report. **100% offline. No account. No cloud.**

Proposed Solution-Rakshak2.0

One-Sentence Summary

A local-first desktop checker that loads a brand rule pack, scans a text draft via deterministic checks + local LLM (RAG), shows rule-cited findings with suggested fixes, and exports corrected text + report — all offline.

Agent Architecture (MVP Scope)



What "Agent" Means Here:

The agent behavior is the **review loop**: it selects relevant rules, runs deterministic and contextual checks, produces structured findings, proposes fixes, records human decisions, and uses accepted terminology preferences in later scans. In the MVP, autonomy is intentionally limited to preserve human control.

Critical Distinction — Check Types:

Type	Examples	Reliability	UI Badge
Deterministic	Banned phrases, required disclaimers, exact terminology, color hex, date/cert expiry	High — exact match against configured rule	🟢 "Rule match"
Advisory (LLM+RAG)	Tone alignment, clarity, inclusivity, fair-language heuristics	Context-dependent; can miss/invent	🟡 "AI assessment — verify"

Never says "compliant" or "approved." Only: "No deterministic violations found; X advisory items flagged for review."

Demo Strategy — Proof on Public Brands

Three Hand-Curated Rule Packs (Source-Verified)

Pack	Public Source	Rules	Purpose
Mozilla	mozilla.org/brand + Trademark Policy	12	Validates tone, terminology, visual rules
Mailchimp	styleguide.mailchimp.com	14	Validates voice, accessibility, CAN-SPAM
Acme SaaS	Synthetic (simulates real org)	10	**Primary demo pack** — matches target user's actual guidelines

No affiliation. No logos redistributed. Text-only seeded examples. Neutral placeholders for visuals.

Seeded Test Set (Measurable Protocol)

- **12 violations** (4 per pack): banned phrases, wrong terminology, missing disclaimers, tone drift — each mapped to one expected rule ID
- **20 clean control passages** with no intended violations
- **Target:** Detect ≥11/12 violations (≥90% recall); False discovery proportion <10% (FP / (TP + FP))
- **TP definition:** Finding identifies correct span AND correct rule category
- **FP definition:** Finding on clean passage OR finding tied to wrong rule
- **Measured:** TP/FP/FN, scan time (deterministic vs LLM), hardware, rule-pack version, model tag, context length, offline verification (Wi-Fi off after model setup)

Three Improvements Rakshak 2.0 Will Deliver

1. **Source-Grounded Findings** — Every issue cites a specific rule ID and guideline source URL (no "AI thinks").
2. **Human-Controlled Corrections** — Users accept, edit, or ignore each suggested fix; decisions recorded in local audit log.
3. **Local-First Privacy** — Guidelines, drafts, findings, audit history remain on-device. No content-related network calls during scanning; offline operation verified with Wi-Fi disabled after model setup.

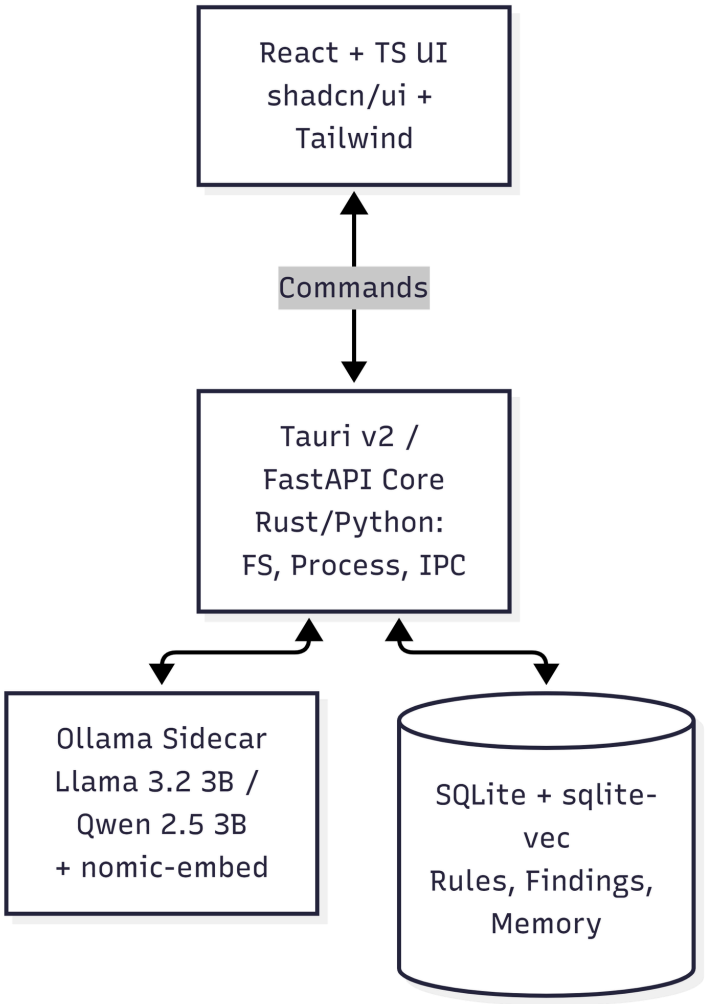
Known Limitations (Honest Boundaries)

- **Language:** English only in MVP.
- **Rule Quality:** Demo uses hand-curated packs; PDF→JSON extraction post-hackathon.
- **Visual Checks:** Not in MVP (logo stretch, color, clear-space are heuristic; P1).
- **PDF Cross-Reference:** Not in MVP (SDS⇔Label, ISI⇔PI; P1).
- **Regulatory Scope: Pre-screening aid only.** Does **not** replace MLR, legal, regulatory, or accessibility approval.
- **Model Hardware:** Llama 3.2 3B / Qwen 2.5 3B for 8–16 GB RAM; Llama 3.1 8B needs 16 GB+ RAM.
- **Install Size:** App installer ~60–100 MB; local models (several GB) managed separately via Ollama.
- **Setup Required:** One-time local Ollama/model installation; no account or cloud service required.

Technology Stack (MVP Only)

Layer	Choice	Rationale
Agent Shell	Tauri v2 (Rust + React/TypeScript)	8 MB binary, native FS, sidecar mgmt
Local LLM	Ollama + `llama3.2:3b-instruct-q4_K_M` / `qwen2.5:3b-instruct`	Runs 8–16 GB RAM, no GPU
Embeddings	`nomic-embed-text` (via Ollama)	Fast 768-dim RAG retrieval
Vector Store	sqlite-vec (SQLite extension)	Zero-dep, embedded, <100 ms
Deterministic Checks	Regex + Rust (phrases, terms, dates, required elements)	High-confidence for explicit rules
Diff/Export	`diff-match-patch`, `docx`, `pdf-lib`	Mature, MIT
State/Storage	Zustand + SQLite (better-sqlite3)	Reactive UI + durable memory
UI	shadcn/ui + Tailwind + Lucide	Accessible, copy-paste components
Packaging	`tauri build` → `.msi` / `.dmg` / `.AppImage`	One command, real installers

if Tauri/Ollama integration unstable by Hour 10 will ship it a local web app using **FastAPI + React** with Ollama running on localhost. Core governance workflow and offline privacy claim remain intact.



Existing solution Comparison

Solution	Checks at creation time	User-owned rules	Local/private	Cited findings	Human-controlled fixes
Brand portals (Bynder, Frontify)	No	Partial	No	No	No
AI writing assistants (Writer, Jasper)	Partial	Limited	No	Often weak	Partial
ChatGPT / Custom GPT	Manual	Manual	No	Unreliable	No
MLR/compliance workflow (Veeva)	No — post-creation	Yes	Enterprise-dependent	Partial	Yes
Rakshak 2.0 MVP	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

Why Now / Why Us

Why Now: AI-generated content volume ↑, distributed teams ↑, privacy regulations ↑ → creation-time local governance is timely.

Why Us: Chemical engineering background (GHS/SDS/regulatory rigor) + hackathon project experience (dashboard/workflow/system design) = domain credibility for future regulated packs + execution capability for this build.

Responsibilities (Solo Developer)

Subsystem	Approach
Rule Packs	Hand-curate 3 JSON packs from cited public sources; validate against seeded violations
Scan Engine	Deterministic regex layer + RAG/LLM for tone/context; check_type badge per finding
UI/UX	3 screens: Onboard (pack selector), Scan (paste + findings), Report (diff + export)
Memory	SQLite: `rules`, `findings`, `user_preferences` (opt-in), `audit_log`
Packaging & Demo	`tauri build` (or FastAPI fallback); test installer; 90-sec screen capture; 5 judge Q&A prep
Documentation	This PDF + `README.md` + architecture diagram + MIT license

---

Development Plan — Round 2

Sprint	Hours	Goal	Tangible Output
Foundation	0–10	Tauri/FastAPI boots; Ollama sidecar; Pack selector loads 3 JSON packs	Pack selector → rules panel
Core Scan	10–26	Deterministic checks working; Findings panel with rule ID, explanation, fix, check_type	Paste draft → findings appear
Review & Export	26–38	Side-by-side diff UI; Accept/Ignore per finding; "Accept All"; Export clean text + `report.html`	End-to-end workflow works
LLM Layer	38–44	RAG + LLM advisory review integrated; confidence badges	Advisory findings appear
Polish & Demo	44–48	Seed test verified; 90-sec capture recorded; installer tested; Q&A ready	**Shippable demo + video**

Buffer: 2 hrs for model download / merges / Wi-Fi.
Stretch (if time): Second pack demo, chat interface. **Cut first if behind:** LLM layer, Pack 2/3.

Evaluation Targets

Metric	Target	Measurement Method
Deterministic Scan Speed	< 2 sec / asset	Seeded test set, local timer
LLM Review Speed	Batched with visible progress	5 assets, progress UI
Recall on Seeded Set	≥ 11/12 violations (≥90%)	TP / (TP + FN)
False Discovery Proportion	< 10%	FP / (TP + FP) on clean + seeded
Offline Operation	0 content-related network calls	Network monitor, Wi-Fi off after model setup
Installer Size (app only)	~60–100 MB	File properties

Future Scope (Post-Hackathon)

Phase	Scope	Trigger
v0.2	VS Code / Figma / Chrome plugins (same agent core)	"Can it work in my tool?"
v0.3	Encrypted team vault via Syncthing/iCloud/Drive (no server)	Small teams want shared memory
v1.0	Optional SaaS dashboard for admins (opt-in)	Enterprise asks for fleet visibility
Community Packs	Shareable rule packs: "Pharma MLR," "Chem GHS/REACH," "Fintech FINRA"	Users contribute back

Core principle: Local-first, privacy-by-default. Cloud features only with explicit opt-in.



Team & Participant Details

Field	Value
Team Name	**ssshariharan82**
Project Name	**Rakshak 2.0**
Team Leader	Hariharan S
Leader Email	`ssshariharan82@gmail.com`
Team Members	Hariharan S
College	St. Joseph's College of Engineering
Drive Link	FutureStack Hackathon - Google...